

AWiR – ćwiczenie 8

Temat:

Powiadomienia e-mail po rejestracji lub modyfikacji użytkownika (Spring Mail).

Cel ćwiczenia:

Rozszerz aplikację z **Ćwiczeń 4-7** o wysyłanie **powiadomień e-mail**:

- po **utworzeniu nowego użytkownika** (rejestracja),
- po **modyfikacji istniejącego użytkownika**.

Powiadomienia mogą być wysyłane:

- do samego użytkownika (na jego adres e-mail z pola email),
- oraz/lub na adres administratora (konfiguracja w properties).

Niezbędne narzędzia:

1. DE: STS 4 / IntelliJ / VS Code
2. Maven 3.9+
3. JDK 21
4. Spring Boot 3.3.x
5. Serwer SMTP do testów

Dotychczasowy projekt zawierał:

Masz już projekt z ćw. 4-7:

- encja User, UserRepository, UserService,
- kontroler MVC (/users/**) i REST (/api/users/**),
- Security (REST = Basic, stateless; MVC = formLogin + remember-me + ewentualnie Google OAuth2),
- ewentualnie SOAP Web Service (ćw. 7).

W tym ćwiczeniu dodaj:

- zależność **spring-boot-starter-mail** w pom.xml,
- konfigurację SMTP w application.properties (np. ZUT),
- serwis MailService (lub UserNotificationService) wykorzystujący JavaMailSender,
- wywołania wysyłki maili w miejscach, gdzie użytkownik jest tworzony lub modyfikowany,
- (opcjonalnie) szablony wiadomości (np. prosty HTML z Thymeleaf lub klasyczny tekst).

Zmiany w strukturze projektu (pogrubione)

```
src/main/java/edu/zut/awir/  
├─ AwirApplication.java  
├─ config/  
│   └─ SecurityConfig.java  
│       └─ WebServiceConfig.java  
├─ model/  
│   └─ User.java  
├─ repository/  
│   └─ UserRepository.java  
└─ service/
```

```

├── UserService.java
├── MailService.java # NOWE: serwis do wysyłania maili
├── web/
├── mvc/
│   ├── UserController.java # tu wywołujemy powiadomienia przy operacjach MVC
├── rest/
│   ├── UserRestController.java # tu wywołujemy powiadomienia dla REST
│   └── RestExceptionHandler.java
├── soap/
└── UserEndpoint.java # (opcjonalnie) można też dodać powiadomienia dla SOAP

src/main/resources/
├── application.properties
├── templates/
├── login.html
├── user-list.html
├── add-user.html
├── edit-user.html
└── mail-user-created.txt # (opcjonalnie) prosty szablon tekstowy

```

1. Rozszerzenie zależności w pliku pom.xml

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>

```

2. Konfiguracja SMTP w application.properties

```

spring.mail.host=sandbox.smtp.mailtrap.io
spring.mail.port=2525
spring.mail.username=YOUR_MAILTRAP_USERNAME
spring.mail.password=YOUR_MAILTRAP_PASSWORD
spring.mail.protocol=smtp
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true

```

```

# Adres administratora - do powiadomień
app.mail.admin=admin@example.com

```

3. Implementacja serwisu mailowego

```

src/main/java/edu/zut/awir/service/MailService.java
package edu.zut.awir.service;

import edu.zut.awir.model.User;
import lombok.RequiredArgsConstructor;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class MailService {

    private static final Logger log = LoggerFactory.getLogger(MailService.class);

    private final JavaMailSender mailSender;

    @Value("${app.mail.admin:admin@example.com}")

```

```

private String adminAddress;

public void sendUserCreated(User user) {
    String subject = "Nowy użytkownik zarejestrowany";
    String text = "Utworzono nowego użytkownika:\n" +
        "ID: " + user.getId() + "\n" +
        "Imię i nazwisko: " + user.getName() + "\n" +
        "Email: " + user.getEmail();

    send(adminAddress, subject, text);

    // kopia do samego użytkownika
    if (user.getEmail() != null && !user.getEmail().isBlank()) {
        send(user.getEmail(), "Witaj w systemie", "Twoje konto zostało utworzone.");
    }
}

public void sendUserUpdated(User user) {
    String subject = "Dane użytkownika zmodyfikowane";
    String text = "Zmodyfikowano dane użytkownika:\n" +
        "ID: " + user.getId() + "\n" +
        "Imię i nazwisko: " + user.getName() + "\n" +
        "Email: " + user.getEmail();

    send(adminAddress, subject, text);
}

private void send(String to, String subject, String text) {
    try {
        SimpleMailMessage msg = new SimpleMailMessage();
        msg.setTo(to);
        msg.setSubject(subject);
        msg.setText(text);
        mailSender.send(msg);
        log.info("Wysłano e-mail do {}: {}", to, subject);
    } catch (Exception e) {
        log.error("Błąd wysyłania e-maila do {}: {}", to, e.getMessage(), e);
    }
}
}

```

4. Wywołanie powiadomień z kontrolerów / serwisu

Tam, gdzie już teraz tworzysz / modyfikujesz użytkownika, po `save()` wywołaj odpowiednią metodę `MailService`.

5. Testowanie

- Uruchom aplikację z poprawną konfiguracją SMTP.
- Zarejestruj nowego użytkownika:
 - przez formularz MVC (`/users/add`), lub
 - przez REST (POST `/api/users`).
- Sprawdź skrzynkę (Mailtrap / MailHog) – powinien pojawić się e-mail o nowym użytkowniku.
- Zmień dane istniejącego użytkownika – powinien przyjść e-mail o modyfikacji.

6. Zadania do wykonania

- Dodaj zależność `spring-boot-starter-mail` i skonfiguruj SMTP w `application.properties`.
- Zaimplementuj `MailService` z metodami `sendUserCreated(User)` i `sendUserUpdated(User)`.
- Wywołaj `MailService` w kontrolerach (MVC i/lub REST) po utworzeniu oraz modyfikacji użytkownika.
- Przetestuj działanie, korzystając z serwera SMTP.

7. Zadanie dodatkowe

- Wykorzystaj HTML w wiadomościach (`MimeMessageHelper`) i dodaj prosty szablon (np. `Thymeleaf`).